

Lorenzo Zoccadelli

Report attività progettuale: studio dell'efficacia di metodi di federated unlearning

Sicurezza dell'informazione

Laurea Magistrale in Ingegneria Informatica

1 Introduzione

A seguito dell'aumento nella diffusione e raccolta di dati e informazioni personali, le legislazioni di tutto il mondo hanno iniziato a regolamentarne condivisione e utilizzo, garantendo maggiori diritti ai propri cittadini per preservarne il più possibile la privacy (nell'UE con l'entrata in vigore del GDPR nel 2016). Si rende perciò necessario garantire tali diritti, tra cui il diritto all'oblio, in tutte le applicazioni che fanno largo uso di dati, tra le quali in particolare vi è l'addestrare modelli di machine learning. Federated Learning (FL) è un metodo di addestramento di reti neurali distribuito che permette di fare il training di un modello senza che i dati grezzi degli utenti lascino i loro dispositivi. La conoscenza relativa a tali dati, però, rimane comunque impressa all'interno del modello (in modo particolare in caso di overfitting), anche se non direttamente accessibile. Per questo motivo è necessario dare all'utente la possibilità di richiedere la cancellazione dei propri dati dall'interno del modello, per garantirgli la possibilità di far valere il suo diritto all'oblio. Ciò porta alla necessità di introdurre anche metodi di Federated Unlearning (FU) che permettano di eliminare il contributo di tali utenti in maniera più efficiente di un semplice retraining da zero del modello.

1.1 Federated Learning e Unlearning

Nel FL, assumendo un'architettura con un server centrale, il training del modello avviene tramite un certo numero di round di comunicazione tra server e client. A ogni round il server invia il modello globale a un sottoinsieme dei client, i quali effettuano un training locale sui propri dati privati e inviano il modello così aggiornato al server. A quel punto il server aggregnerà tutti gli aggiornamenti ricevuti, ottenendo la nuova versione modello.

FedAvg è uno dei metodi più utilizzati: ogni client esegue un certo numero di epoche locali e invia al server il modello così aggiornato. L'aggregazione lato server avverrà semplicemente effettuando la media pesata di tutti i modelli ricevuti.

Un approccio alternativo prevede invece che i client condividano direttamente i gradienti. In questo caso, il server li aggrega (ad esempio facendone la media) e li applica lui stesso al modello globale. Tuttavia, questo approccio può comportare maggiori costi di comunicazione e una potenziale maggiore vulnerabilità ad attacchi di tipo gradient leakage che sfruttano i gradienti per ricostruire i batch di dati che li hanno generati [10].

L'obiettivo del FU invece è quello di eliminare il contributo di specifici dati dalla rete (nell'ambito dell'attività progettuale tutti i dati forniti da uno specifico client), sottostando ai vincoli della configurazione federata, ovvero senza alcun accesso diretto ai dati degli utenti. Idealmente si vorrebbe generare un modello statisticamente identico a uno addestrato da zero senza includere il client che ha fatto la richiesta di unlearning (retraining). Questo approccio risulta ovviamente in un unlearning esatto, in cui si ha la certezza a priori che il modello finale non sia stato influenzato in alcun modo dai dati "dimenticati". Tale soluzione è però altamente dispendiosa in quanto significherebbe perdere il lavoro fatto in precedenza per creare il modello iniziale e pagare l'intero costo di un nuovo training. Sebbene vi siano altri metodi di unlearning esatto (come l'adattamento di SISA [1] in ambito federato), essi rimangono computazionalmente costosi, introducendo spesso un trade-off tra costo computazionale e di comunicazione e occupazione in memoria legata al salvataggio di un ensemble di modelli o di checkpoint durante il training.

Altri metodi invece, si propongono di eseguire un unlearning approssimato modificando il modello già addestrato ed eseguendo al più qualche round di recupero delle prestazioni, garantendo maggiore efficienza ma minori garanzie. Questi metodi sfruttando tecniche diverse [7], tra cui: riaddestramento più rapido basato sulla calibrazione dello storico degli update (FedEraser [6]), esecuzione di epoche di gradient ascent mirato ai dati da dimenticare sul client che ne fa la richiesta [4], eliminazione dei contributi e recupero delle prestazioni tramite knowledge distillation [9], o semplice degradazione della rete seguita da epoche di recupero delle prestazioni (come NoT [5]). In tutti questi casi però, anche per via della complessità intrinseca dei modelli di deep learning, non ci sono garanzie a priori che il modello si comporti come se quei dati non fossero mai stati visti durante il training iniziale.

1.2 Motivazioni dell'attività progettuale

Diventa quindi cruciale di fronte a vari approcci di unlearning approssimato, avere metodi efficaci per valutarne la reale capacità nel "dimenticare" le informazioni. Tale valutazione non è banale in quanto spesso, al lato pratico al di là degli aspetti teorici di somiglianza statistica al modello retrained, non è chiaro cosa si intenda dicendo che "la rete ha veramente dimenticato un dato". La maggior parte di questi algoritmi di unlearning viene valutata confrontando le prestazioni (accuracy, f1-score...) sui dati unlearned, di training e di test, e su attacchi di tipo membership inference, che sfruttano sempre l'output finale della rete o la loss calcolata a partire da esso. Ma, data la tendenza delle reti neurali a creare rappresentazioni interne dei dati in modo gerarchico tra i layer, questi test possono risultare limitativi in quanto non valutano le possibili influenze dei dati di training latenti all'interno della rete nei livelli intermedi. Lo scopo di questa attività progettuale è quindi quella di indagare come poter valutare in maniera efficace l'effettivo unlearning (del contributo dell'intero dataset di un client), e nello specifico in che modo il contributo dei dati di training (e quindi possibilmente di quelli su cui viene fatto l'unlearning) sia rilevabile anche all'interno della rete e non solo nel suo output.

2 Studio dell'Efficacia dell'Unlearning

Con l'obiettivo di analizzare l'efficacia dei metodi di unlearning nella maniera precedentemente descritta, sono stati fatti diversi esperimenti per rilevare differenze nel comportamento tra il primo modello iniziale (addestrato con FedAvg), lo stesso modello addestrato in maniera analoga escludendo il client specifico che richiede l'unlearning (retrained) e due metodi di unlearning approssimato: Gradient Ascent (GA [4]) e NoT [5]. Per tutti i test (a meno che non sia specificato diversamente), è stata utilizzata una ResNet18 addestrata su CIFAR10, modificando la risoluzione delle immagini da 32x32px a 224x224px. Sono stati utilizzati 10 client ai quali sono stati assegnati 1000 immagini ognuno (per un totale di 10000 immagini di training).

In letteratura alcuni algoritmi di unlearning vengono testati nella loro efficacia nell'eliminare il contributo di una backdoor iniettata nei dati di training. Diversamente, i test e le analisi condotte sono state fatte su dati IID e in uno scenario in cui l'unlearning viene eseguito per motivi di privacy e non come difesa per eliminare il contributo di una backdoor. Infatti, alcuni algoritmi sono efficaci nel secondo caso, in cui spesso è sufficiente rendere inefficace il pattern malevolo specifico, ma ottengono risultati peggiori nel primo più generico scenario.

I risultati di accuracy ottenuti dai vari modelli sono riportati nella tabella 2.1 da cui si può già notare come GA ottenga prestazioni migliori per i dati unlearned rispetto a quelli di test, mostrando un comportamento diverso tra i due dataset. L'ampio overfitting ottenuto dai modelli è voluto in quanto spesso è una delle cause più importanti che determina un diverso comportamento della rete tra dati di training rispetto a quelli di test.

Table 2.1: Statistiche di accuracy ottenute dai vari modelli

Modello	Train Acc.	Unl. Acc.	Test Acc.
FedAvg	0.999	0.997	0.601
FedRetrain	0.999	0.580	0.592
GA	0.999	0.838	0.605
NoT	0.996	0.611	0.619

2.1 LiRA come Membership Inference Attack

Una metrica comunemente utilizzata per valutare l'unlearning è il tasso di successo di un membership inference attack (MIA). Nello specifico, si valuta la capacità dell'attacco di distinguere

correttamente i dati soggetti a unlearning da generici dati di test, classificando campioni estratti da un insieme che comprende entrambi. Generalmente, questi attacchi sfruttano le discrepanze negli output prodotti dalla rete quando in input riceve dati di training (unlearned in questo caso) o di test, concentrandosi in particolar modo sull'analisi dei valori della loss. Nel caso ideale in cui la rete si comporti esattamente allo stesso modo sia per i dati dimenticati sia per quelli di test, l'attacco dovrebbe comportarsi come un classificatore casuale. Per condurre questa valutazione, viene spesso utilizzato un semplice classificatore binario addestrato direttamente sugli output del modello bersaglio, oppure si ricorre a tecniche più complesse basate su shadow model, di cui la più famosa introdotta in [8]. Tuttavia, come evidenziato in [3], esistono metodologie più avanzate ed efficaci, come LiRA [2]. Quest'ultimo approccio migliora l'accuratezza dell'attacco evitando di utilizzare una singola soglia globale, ma analizzando, per ogni singolo campione, le distribuzioni degli output generati da una serie di shadow model (addestrati, rispettivamente, includendo o escludendo il dato in esame dal loro training set).

Per queste ragioni in primo luogo sono stati valutati tutti i modelli attraverso un MIA basato su LiRA (con 64 shadow model) in modo da rilevare se già nell'output della rete vi siano differenze significative per quanto riguarda i dati dimenticati. Idealmente, dopo l'unlearning, il modello si dovrebbe comportare in modo paragonabile a un classificatore casuale ma, come estensivamente motivato in [2], la sola accuracy potrebbe non essere sufficiente per valutare in modo significativo l'efficacia dell'attacco. L'analisi è quindi stata condotta analizzando le ROC curve create a partire dai punteggi ottenuti con LiRA, mettendo in risalto tramite scala logaritmica anche il TPR in regime di bassi FPR. I risultati sono riportati in Figura 2.1, dove si può notare come GA mostri un leggero miglioramento rispetto al modello originale, ma è evidente come il contributo dei dati dimenticati sia ancora pienamente rilevabile. D'altro canto, il risultato su NoT è molto simile a quello ottenuto sul modello retrained.

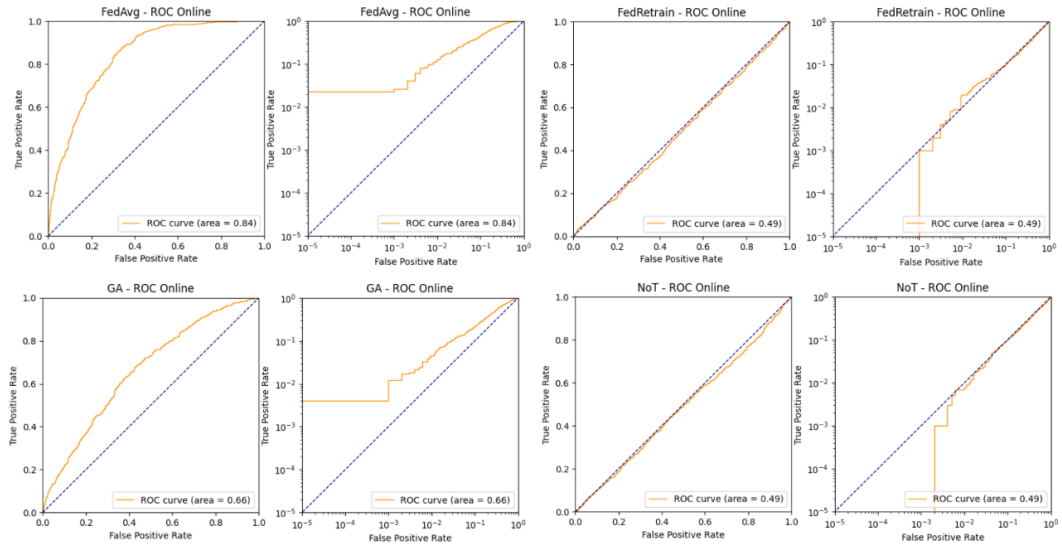


Figure 2.1: ROC curve LiRA per i modelli analizzati

2.2 Analisi delle Feature dell'Ultimo Layer

Le reti neurali durante l'addestramento imparano a classificare i dati di input trasformandoli, attraverso i vari layer, in rappresentazioni interne in grado di catturare feature sempre più specifiche. La rappresentazione ottenuta a valle della rete viene poi classificata attraverso gli ultimi layer. È quindi possibile separare concettualmente la rete in due: una prima parte che esegue l'estrazione delle feature da una seconda che le classifica. In una CNN (come la ResNet18 utilizzata) solitamente la prima è formata dai layer convoluzionali, mentre la seconda è costituita dai layer fully connected successivi. Le attivazioni tra queste due parti (subito dopo il global average pooling nella ResNet18) sono spesso considerate come una rappresentazione vettoriale interna dell'immagine di input in uno spazio ad alta dimensionalità. La rete impara tale rappresentazione durante il processo di addestramento, perciò anche questa rappresentazione intermedia è in principio suscettibile a potenziali differenze tra dati di training e di test, che potrebbero risultare evidenti anche a seguito del processo di unlearning.

Il fatto che la semantica di ogni elemento di questi vettori è appresa in modo automatico dalla rete e che ogni possibile versione della rete può cambiare tale rappresentazione (anche nel caso in cui una rete sia semplicemente addestrata per epoche aggiuntive), rende complicato confrontare le rappresentazioni dello stesso dato prodotte da due o più reti diverse. Per fare questo tipo di confronto la cosa più semplice è stata estrarre statistiche che aggregino le intere rappresentazioni vettoriali come norma, media, valore massimo (all'interno del vettore) e varianza. L'idea alla base è quella che una rete che ha visto un certo esempio durante il training lo riesca a rappresentare in maniera più precisa, risultando in valori più alti per le feature più significative per quell'esempio e valori più vicini allo zero per quelle non presenti (qualunque sia la semantica assegnata dalla rete alle features). Al contrario per dati mai visti la rete potrebbe essere più incerta sulla loro rappresentazione, assegnando valori più simili tra feature effettivamente presenti e non. Seguendo questo approccio sono state riscontrate differenze apprezzabili solamente considerando il massimo e la varianza delle feature. In Figura 2.2 sono riportati i risultati ottenuti sui vari modelli come distribuzioni delle statistiche per membri (dati unlearned) e non membri (dati di test) e le ROC curve ottenute eseguendo LiRA sulla base di queste statistiche invece che sulla loss. In questo caso non si rilevano differenze nei due metodi di unlearning se non una curva leggermente sopra a quella ottenibile da un classificatore casuale per quanto riguarda GA.

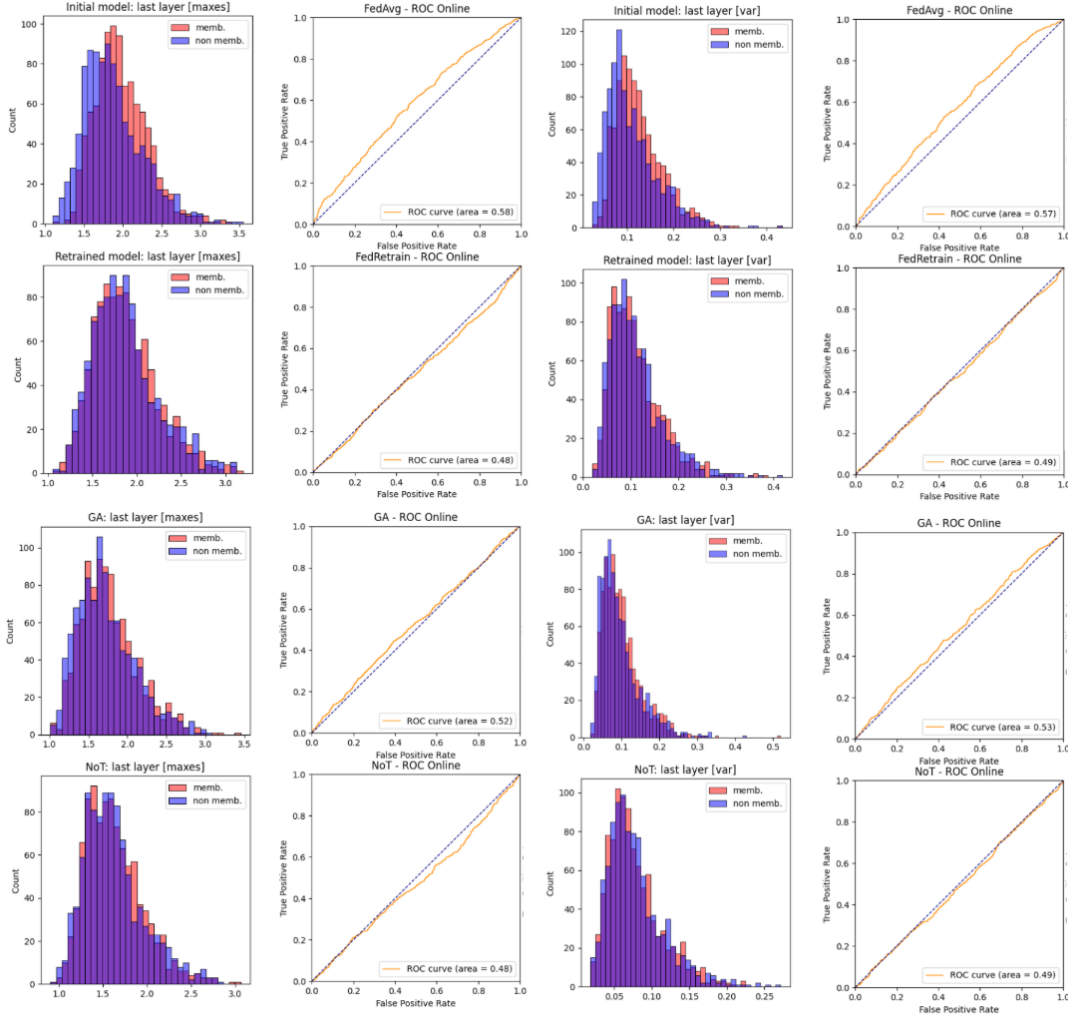


Figure 2.2: Distribuzioni del massimo e della varianza delle feature estratte dalla parte convoluzionale della rete. Sono mostrate le distribuzioni per membri (dati unlearned) e non membri (dati di test). Si nota come praticamente solo per FedAvg e molto leggermente per GA la differenza sia evidente

Risultati migliori si ottengono invece se si analizzano le feature (estratte da una stessa rete) proiettandole in uno spazio 2D. L'idea, ancora una volta, risiede nel fatto che una rete che ha visto i dati durante il training li rappresenterà in maniera “più precisa”, dando origine a cluster più densi di quelli che si otterrebbero per dati di test. Per visualizzare questi cluster in maniera qualitativa in uno spazio bidimensionale i metodi migliori si sono rivelati essere t-SNE e UMAP (per brevità i risultati sono riportati solo per la rappresentazione con UMAP Figura 2.3). In questo caso la differenza tra i cluster dei dati unlearned (membri) e dei dati di test (non membri) è palese nel caso di FedAvg, dove i primi creano cluster ben definiti mentre i secondi risultano essere più sparsi, mentre in FedRetrain ovviamente le due categorie sono indistinguibili. GA e NoT si pongono invece in una via di mezzo: i cluster dei membri sono leggermente più definiti, ma le differenze ad occhio nudo sono meno evidenti (in GA si notano leggermente meglio, mentre in NoT sono minime). Per quanto riguarda GA, però, se si calcolano

per ogni classe il baricentro dei membri e dei non membri, e si prendono le distanze di ogni punto da tali baricentri Figura 2.4, le differenze diventano più nette, soprattutto considerando le proiezioni ottenute con UMAP Figura 2.5.

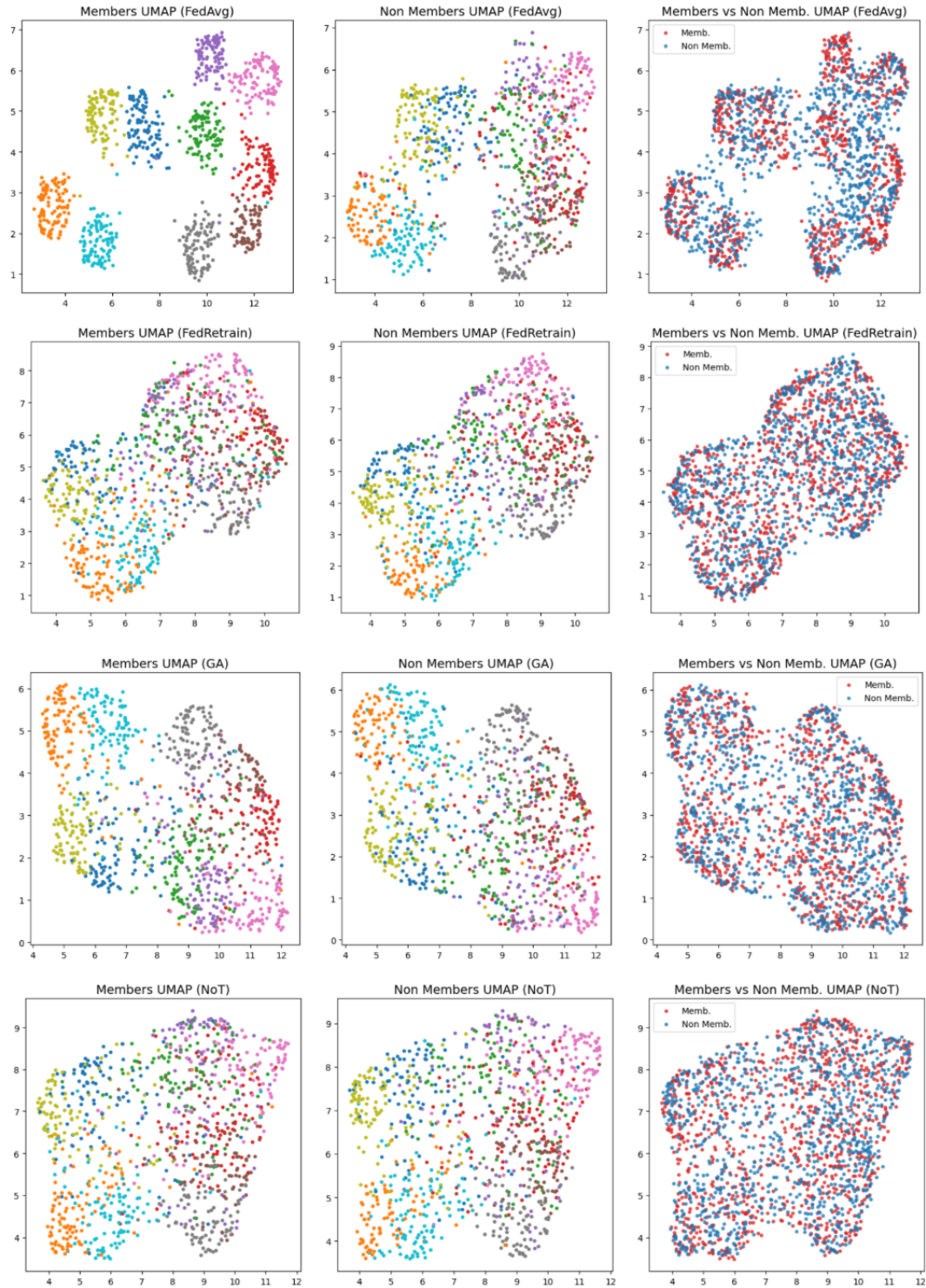


Figure 2.3: Proiezione di membri e non membri per tutti i modelli analizzati

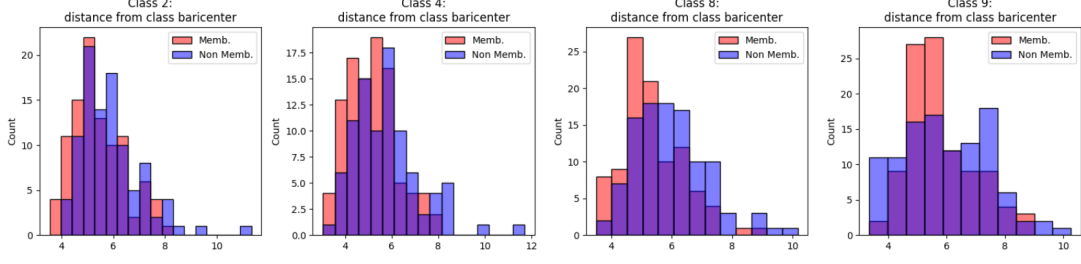


Figure 2.4: Distribuzione delle distanze di membri e non membri dai rispettivi baricentri considerando lo spazio delle feature

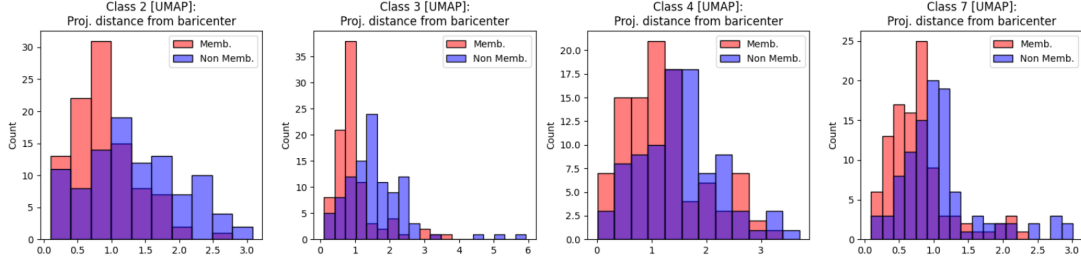


Figure 2.5: Distribuzione delle distanze di membri e non membri dai rispettivi baricentri considerando la proiezione 2D con UMAP

2.3 Ricostruzione Immagini Tramite Attivazioni

Lo studio del comportamento dei layer intermedi invece risulta essere più complicato. Tale difficoltà deriva dall'elevata dimensionalità delle attivazioni (nelle CNN, considerando la totalità dei canali, il numero di singoli valori di attivazione cresce in modo significativo) e dalla maggiore difficoltà nel rilevare le differenze tra esse più ci si avvicina all'input. In tal caso, più si considerano layer vicini all'input, più tali layer cercano feature generiche, meno dipendenti dagli esempi specifici su cui viene fatto l'unlearning.

D'altro canto, i layer intermedi possono comunque essere sfruttati partendo dall'idea che più si va in profondità nella rete, più nelle attivazioni si perdono informazioni riguardanti l'input iniziale, in quanto il modello impara a “selezionare” quelle più utili da mantenere. Perciò è probabile, soprattutto in caso di overfitting, che durante il forward pass una rete mantenga internamente, in certi layer, più informazioni per immagini che ha visto durante il training rispetto a immagini di test.

Questa idea può essere sfruttata cercando di ricostruire un'immagine in base alla sua attivazione in un determinato layer. Ciò può essere realizzato eseguendo il forward pass dell'immagine da ricostruire, salvandone l'attivazione a un layer specifico scelto. Successivamente si fornisce alla rete un'immagine fittizia, tipicamente inizializzata con rumore casuale o costituita da un pattern neutro (ad esempio un'immagine completamente nera). Mantenendo i pesi del modello congelati, si procede poi all'ottimizzazione iterativa dei valori dei singoli pixel del nuovo input (in modo analogo all'ottimizzazione dei pesi durante il training, ad esempio tramite SGD o

Adam) al fine di minimizzare la distanza tra l'attivazione generata dall'immagine fittizia e quella originale precedentemente salvata. Qualora le rappresentazioni interne del layer scelto abbiano preservato una quantità sufficiente di informazioni relative all'immagine originale, al termine del processo l'immagine fittizia conterrà una ricostruzione dell'immagine di partenza.

Dalla Figura 2.6 si nota come in molti esempi il modello originale (FedAvg) riesca a ricostruire l'immagine di partenza meglio del modello retrained. Per quanto riguarda metodi di unlearning testati, NoT a volte si comporta in maniera simile al modello retrained, ma altre riesce a ricostruire l'immagine in maniera più nitida, come FedAvg, mostrando come sia probabile che il modello mantenga alcune informazioni in più sull'immagine originale. Per GA invece la ricostruzione è quasi sempre più simile a quella di FedAvg, ottenendo spesso l'immagine completa.

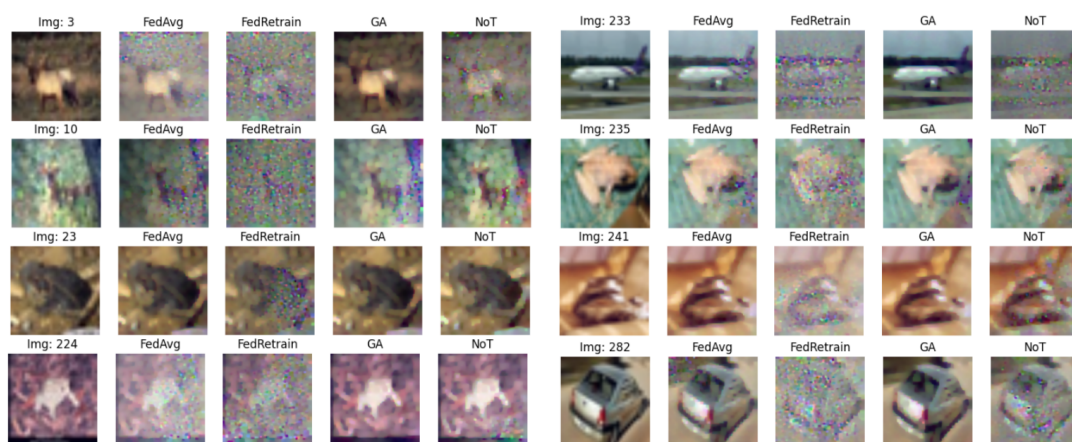


Figure 2.6: Immagini unlearned ricostruite prendendo l'attivazione del primo layer convoluzionale del secondo residual block del quarto stage della ResNet18

In Figura 2.7 sono riportati anche alcuni esempi ottenuti dalla ricostruzione delle immagini a partire dall'attivazione al termine del secondo stage della ResNet18. In questo caso per ottenere un risultato migliore, la dimensione delle immagini è stata mantenuta a 32x32px e la rete adattata a tale modifica. In questo caso la differenza non sta tanto nella riconoscibilità dell'immagine, ma piuttosto nella quantità di rumore rimasto nell'immagine ricostruita.



Figure 2.7: Immagini unlearned ricostruite (32x32px) prendendo l'attivazione al termine del secondo stage della ResNet18

Va specificato come, per quanto riguarda questo processo, i test effettuati sono fortemente

legati a vari fattori come i parametri dell'algoritmo di ricostruzione, le caratteristiche delle immagini da ricostruire e da come i modelli organizzano internamente le informazioni estratte dalle immagini. Gli esempi mostrati sono stati selezionati tra quelli in cui i risultati sono stati più evidenti, ma per la maggior parte delle immagini le differenze non risultavano essere così marcate.

3 Conclusioni

Si può quindi concludere che le reti neurali (in questo caso specifico le CNN per la classificazione di immagini) tendono a mantenere informazioni relative ai dati utilizzati per il training anche all'interno dei propri livelli interni, soprattutto in caso di overfitting. Queste informazioni sono codificate nei valori dei pesi e, sebbene l'estrazione, l'interpretazione e lo sfruttamento della loro semantica non siano immediati, i loro effetti possono essere rilevati anche nei livelli intermedi, diventando più evidenti più si va in profondità nella rete. Per queste ragioni la valutazione dell'efficacia di un metodo di unlearning approssimato potrebbe essere limitata se si considera solamente l'output finale per analizzarne il comportamento sui dati "dimenticati". L'analisi delle metriche di performance (accuracy, f1-score...) e la valutazione della riuscita di membership inference attack (ad esempio tramite LiRA) potrebbero essere integrati con metodologie incentrate sull'ispezione delle dinamiche interne, focalizzando l'attenzione anche sulle feature estratte dalla sezione convoluzionale e fornite in input al classificatore (analizzandone per esempio le distribuzioni) e sfruttando le attivazioni dei layer intermedi (ad esempio attraverso la ricostruzione delle immagini oppure con strumenti come Grad-CAM per analizzare le porzioni dell'immagine più importanti per la classificazione a un certo layer). Ovviamente è probabile che possano esistere metodi più efficaci per trovare differenze nel comportamento della rete a seguito dell'unlearning rispetto a una riaddestrata da zero, ma in Figura 3.1 è rappresentato lo schema di un possibile framework per l'analisi dell'efficacia di un algoritmo di unlearning che può risultare da quello che è stato fatto in questa attività progettuale.

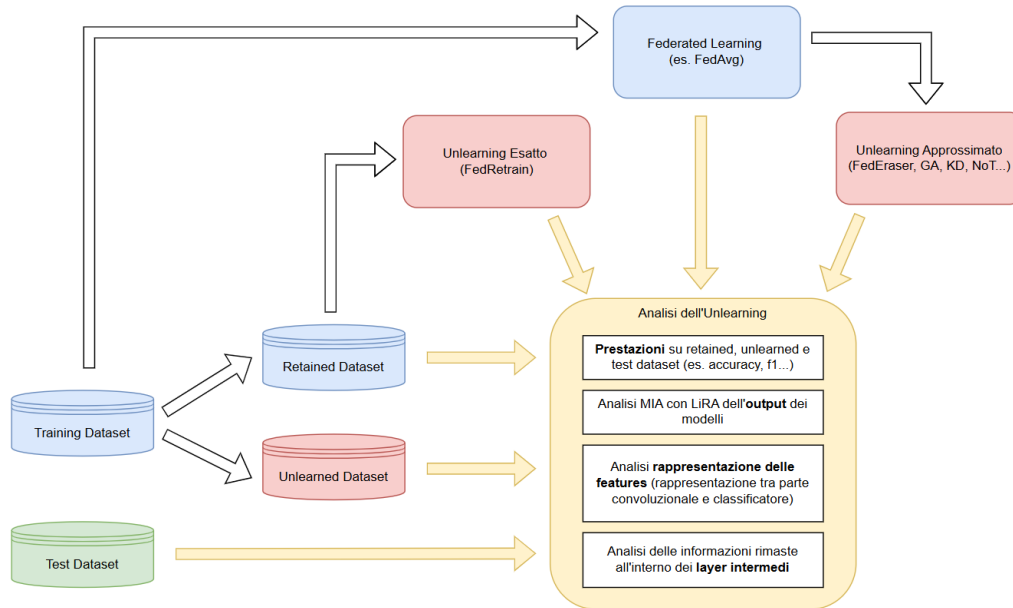


Figure 3.1: Idea di framework realizzabile mettendo insieme tutte le considerazioni fatte in questa attività progettuale

In questo modo è possibile valutare il modello ottenuto dopo la fase di unlearning, confrontandolo con quello originale e con un modello retrained, analizzandolo su vari livelli: sulla base delle prestazioni, dell'output, della rappresentazione vettoriale interna degli input e delle attivazioni dei layer intermedi.

Per quanto riguarda i modelli analizzati, si può notare come GA, a discapito della sua efficienza (dopo la fase di ascesa del gradiente bastano solamente poche epoche per recuperare le prestazioni), in realtà lascia molte informazioni all'interno della rete. Al contrario NoT sembra essere molto più efficace, nonostante nella ricostruzione delle immagini tende a comportarsi spesso in maniera simile al modello originale, ma richiede molte più epoche per recuperare le prestazioni, rendendolo a livello di efficienza paragonabile a un retraining da zero.

Repository github contenente tutto il codice dell'attività progettuale: https://github.com/zocca02/federated_unlearning_project

References

- [1] Lucas Bourtoutle, Varun Chandrasekaran, Christopher A Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. “Machine unlearning”. In: *2021 IEEE symposium on security and privacy (SP)*. IEEE. 2021, pp. 141–159.
- [2] Nicholas Carlini, Steve Chien, Milad Nasr, Shuang Song, Andreas Terzis, and Florian Tramèr. “Membership inference attacks from first principles”. In: *2022 IEEE symposium on security and privacy (SP)*. IEEE. 2022, pp. 1897–1914.
- [3] Keltin Grimes, Collin Abidi, Cole Frank, and Shannon Gallagher. “Gone but not forgotten: Improved benchmarks for machine unlearning”. In: *arXiv preprint arXiv:2405.19211* (2024).
- [4] Anisa Halimi, Swanand Kadhe, Ambrish Rawat, and Nathalie Baracaldo. “Federated unlearning: How to efficiently erase a client in fl?” In: *arXiv preprint arXiv:2207.05521* (2022).
- [5] Yasser H Khalil, Leo Brunswic, Soufiane Lamghari, Xu Li, Mahdi Beitollahi, and Xi Chen. “Not: Federated unlearning via weight negation”. In: *Proceedings of the Computer Vision and Pattern Recognition Conference*. 2025, pp. 25759–25769.
- [6] Gaoyang Liu, Xiaoqiang Ma, Yang Yang, Chen Wang, and Jiangchuan Liu. “Federated unlearning”. In: *arXiv preprint arXiv:2012.13891* (2020).
- [7] Nicolò Romandini, Alessio Mora, Carlo Mazzocca, Rebecca Montanari, and Paolo Bellavista. “Federated unlearning: A survey on methods, design guidelines, and evaluation metrics”. In: *IEEE Transactions on Neural Networks and Learning Systems* 36.7 (2024), pp. 11697–11717.
- [8] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. “Membership inference attacks against machine learning models”. In: *2017 IEEE symposium on security and privacy (SP)*. IEEE. 2017, pp. 3–18.
- [9] Chen Wu, Sencun Zhu, and Prasenjit Mitra. “Federated unlearning with knowledge distillation”. In: *arXiv preprint arXiv:2201.09441* (2022).

- [10] Ligeng Zhu, Zhijian Liu, and Song Han. “Deep leakage from gradients”. In: *Advances in neural information processing systems* 32 (2019).